

Project Overview

This project is designed to enhance a chess game by integrating a locally-hosted, open-source Language Model (LLM) to provide dynamic and engaging narrative descriptions and visual effects for each move. The system uses the Ollama framework to run models such as Llama 3, Mistral, or Qwen on the user's local machine without the need for an API key or cloud services. If the model fails to respond or provides invalid output, the system gracefully falls back to predefined narrations and effects to ensure the game continues smoothly.

Architecture

The architecture is centered around a single function `get_narration`` which interacts with the LLM to generate narrations and effects for chess moves. It includes fallback mechanisms to handle errors or unexpected responses from the LLM. The system is modular, allowing easy integration of different models or changes to the fallback logic.

Key Components

- **LLM Interaction**: The core component that communicates with the locally-hosted LLM to generate narrations and effects.
- **Fallback Mechanism**: Provides default narrations and effects when the LLM response is unavailable or incorrect.
- **Prompt Construction**: Builds the input prompt for the LLM based on the move information.
- **Category Determination**: Determines the category of the move (e.g., checkmate, capture) to select appropriate fallback effects.

Main Functions

- **get_narration(move_info, timeout=8)**: This function takes move information and generates a narration and effect by sending a request to the LLM. It handles exceptions and uses fallbacks if necessary.
- **build_prompt(move_info)**: Constructs the prompt string that will be sent to the LLM, including the move data in JSON format.
- **_fallback_narration(move_info)**: Generates a default narration based on the type of move.
- **_fallback_effect(move_info)**: Selects a default effect from a predefined list based on the move category.
- **_category(move_info)**: Determines the category of the move to help choose the correct fallback effect.

Data Flow

1. **Chess Move Input**: The chess game sends move information to the `get_narration`` function.
2. **Prompt Construction**: The `build_prompt`` function creates a formatted prompt string.
3. **LLM Request**: The `get_narration`` function sends the prompt to the LLM via HTTP POST.
4. **Response Handling**: The response from the LLM is parsed, and the narration and effect are extracted.
5. **Fallback Check**: If the LLM response is empty or contains invalid data, the system uses `_fallback_narration`` and `_fallback_effect``.
6. **Output**: The function returns a dictionary containing the narration, effect, source, and optionally an error message.

Setup Instructions

1. ****Install Dependencies****:

- Ensure Python is installed on your system.
- Install the required Python packages using pip:

```
```bash
```

```
pip install requests
```

```
```
```

2. ****Configure Environment Variables****:

- Set the `OLLAMA\_URL` environment variable to point to your locally-hosted Ollama instance.
- Define the `MODEL\_NAME` environment variable with the name of the model you wish to use (e.g., Llama 3, Mistral, Qwen).

3. ****Define Constants****:

- In your code, define constants such as `PIECE\_NAMES`, `VALID\_EFFECTS`, and `FALLBACK\_EFFECTS`. These should map chess pieces to their names in the RPG context and specify valid and fallback visual effects.

4. ****Run the Application****:

- Integrate the `get\_narration` function into your chess game application to generate narrations and effects for each move.

Future Improvements

- ****Model Training****: Train custom models specifically for generating chess move narrations and effects to improve accuracy and relevance.
- ****Effect Customization****: Allow users to customize the list of effects and their mappings to different move categories.
- ****User Interface****: Develop a user interface to allow users to select different models or adjust parameters like temperature.
- ****Logging****: Implement logging to track requests to the LLM and fallback usage for better debugging and performance analysis.
- ****Error Handling****: Enhance error handling to provide more detailed feedback about issues with the LLM response.